UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ
FACULTAD DE INGENIERÍA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS PROGRAMA DE INGENIERÍA DE SISTEMAS

GUÍA PRÁCTICA SEMANA 07

Asignatura: Desarrollo de Aplicaciones Web (IS093A)
Unidad I: Desarrollo Web Frontend
Tema: React Hooks: useState, useEffect, useContext, useRef, useReducer, useCallback, useMemo y Hooks Personalizados
Modalidad: Presencial / Laboratorio
Fecha: 14.10.2026
Duración: 90 minutos

APELLIDOS Y NOMBRES: Pariona Ramos Saul Anibal

OBJETIVO DE LA PRÁCTICA

Dominar el ciclo de renderizado de React mediante el uso estratégico de **Hooks nativos**, aplicar useEffect para efectos secundarios y limpieza, gestionar estado global con useContext, optimizar rendimiento con useMemo/useCallback, manejar referencias inmutables con useRef, refactorizar lógica compleja con useReducer, y encapsular comportamientos reutilizables en **Hooks Personalizados**. Validar el flujo de re-renders con React DevTools Profiler.

RECURSOS REQUERIDOS

- Node.js LTS (≥20.x) + npm/pnpm
- Visual Studio Code (Extensiones: ES7+ React Snippets, React Developer Tools, ESLint, Error Lens)
- Proyecto base Vite + React (heredado de Semana 06 o scaffold nuevo)
- Navegador moderno + React DevTools (pestaña Profiler y Components)
- Documentación oficial: react.dev/reference/react
- Conexión a internet

PARTE 1: EJERCICIO PASO A PASO

Paso	Actividad	Concepto Clave	Restricción/Nota Técnica
1	Revisar ciclo de render: identificar qué dispara re-render en useState vs props. Configurar React.StrictMode y Profiler	Render cycle, batch updates, doubleinvoke in dev	No usar console.log para medir rendimiento. Usar Profiler

Paso	Actividad	Concepto Clave	Restricción/Nota Técnica
2	Refactorizar fetch de API: migrar .then() a async/await dentro de useEffect, añadir array de dependencias y función de limpieza (AbortController)	Side effects, dependency array, cleanup, memory leaks	Explicar por qué [ ] ≠ "se ejecuta una vez" en StrictMode
3	Implementar useContext para tema global o estado de sesión. Crear Provider y consumirlo sin prop drilling	Context API, Provider pattern, performance impact	Evitar contextos gigantes. Separar por dominio (ThemeContext, AuthContext)
4	Optimizar render: envolver filtros pesados en useMemo, estabilizar callbacks en useCallback, usar useRef para DOM/input focus sin re-render	Referential equality, memoization, DOM refs, avoid overoptimization	Documentar métricas: Why Did You Render? o Profiler commits
5	Extraer lógica repetitiva en **Hook Personalizado** (useFetch o useForm). Validar reglas de Hooks y testear reutilización	Composition over inheritance, Hook rules, encapsulation	Prohibido llamar Hooks dentro de loops, condiciones o funciones anidadas

USO DE HERRAMIENTAS DE IA (Recomendado para optimizar el ejercicio)

Herramienta	Uso Permitido	Prompt Ejemplo	Uso Prohibido
ChatGPT/Claude/Copilot	Explicar dependency array pitfalls, refactorizar a useReducer, depurar useCallback	"¿Por qué mi useEffect se ejecuta en loop infinito al pasar un objeto como dependencia?"	"Genera la app completa con todos los hooks y optimizaciones"
React DevTools Profiler	Medir render commits, identificar	(Extensión oficial)	Usar Profiler solo para "ver si funciona", sin

Herramienta	Uso Permitido	Prompt Ejemplo	Uso Prohibido
	componentes que se renderizan innecesariamente, comparar before/after memo		analizar métricas
ESLint eslint-pluginreact-hooks	Validar reglas de Hooks, detectar dependencias faltantes, advertir sobre antipatrones	(Automático)	Desactivar reglas de lint sin justificación técnica documentada

Regla de laboratorio: El 80% de la implementación de hooks debe escribirse manualmente. Cada intervención de IA para resolver ciclos de render o dependencias debe llevar // IA: [problema] → Solución manual: [explicación] Evidencias del Ejercicio Desarrollado:

- Añada capturas de pantalla de cada paso realizado en Visual Studio Code.

  PASO 1: REVISAR EL CICLO DE RENDERIZADO Y HERRAMIENTAS DE DIAGNÓSTICO

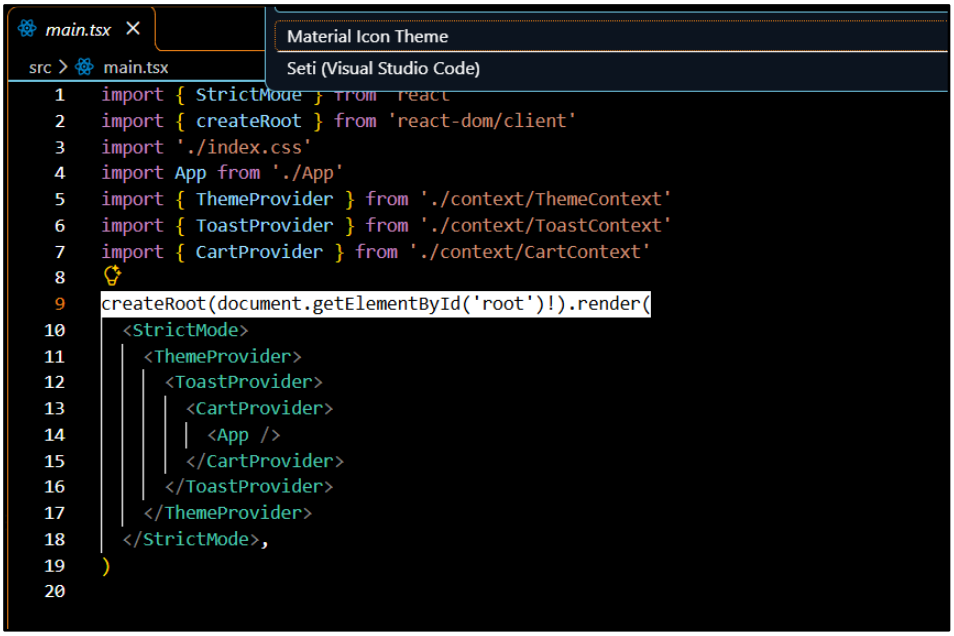
  1.1. Objetivo de la Actividad

  Identificar los desencadenantes de un nuevo renderizado en React, diferenciar el uso de estados y propiedades (props), configurar StrictMode y utilizar las herramientas de diagnóstico React DevTools Profiler.

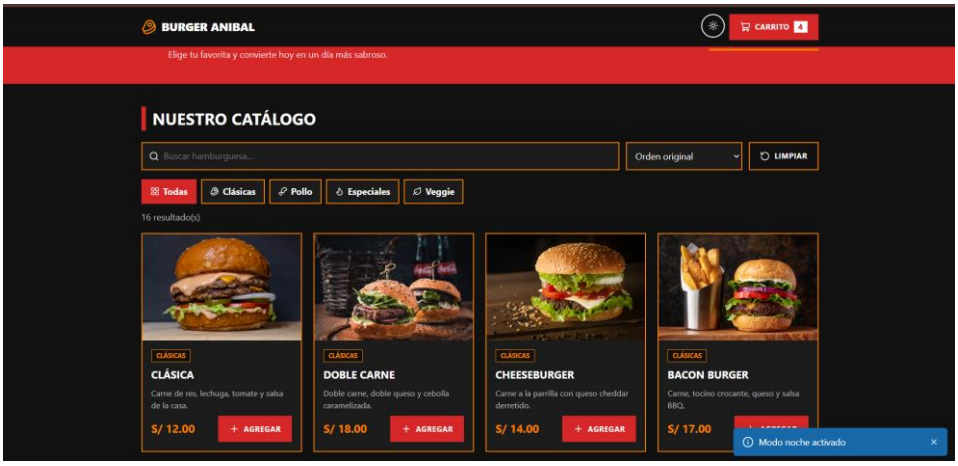
  1.2. Análisis del Código y Conceptos Clave

  - Configuración del Entorno (main.tsx):
    - Se utiliza StrictMode, el cual permite a React detectar problemas relacionados con efectos secundarios y ciclos de vida durante el desarrollo mediante montajes y desmontajes dobles simulados.
    - Se emplea createRoot(document.getElementById('root')!).render(...) para inicializar la aplicación y renderizar el componente principal App.
  - Desencadenantes de Renderizado:

  - Modificaciones en el carrito (agregar, cambiar cantidad, eliminar, abrir/cerrar).
  - Cambios de interfaz y preferencias (cambio de tema día/noche, consultas en el buscador, filtrado por categorías, reordenamiento por precio).
  - Carga inicial de datos asíncronos desde el archivo burgers.json.
- Diferencia entre Estado y Props:
  - Estado (useState): Datos internos propios de un componente que pueden cambiar a lo largo del ciclo de vida (ej. query, cat, orden en Catalog.tsx, o cartOpen en App.tsx).
  - Props: Datos de sólo lectura que un componente recibe desde su componente padre (ej. burgers y onAdd en Catalog.tsx, o burger y onAdd en BurgerCard.tsx).



Desencadenantes de Dia y noche



Mantenimiento de estado al recargar la pagina



PASO 2: GESTIÓN DE EFECTOS, ASINCRONÍA Y LIMPIEZA (useEffect, async/await, AbortController)

2.1. Objetivo de la Actividad

Refactorizar la carga de datos aplicando funciones asíncronas dentro de useEffect, gestionar dependencias correctamente e implementar AbortController para prevenir fugas de memoria y cancelar peticiones obsoletas.

2.2. Análisis del Código y Conceptos Clave

Implementación en useFetch.ts:

```ts
import { useCallback, useEffect, useState } from 'react'

interface FetchState<T> {
  data: T | null
  loading: boolean
  error: string | null
}

export function useFetch<T>(url: string) {
  const [state, setState] = useState<FetchState<T>>({
    data: null,
    loading: true,
    error: null,
  })
  const [reloadKey, setReloadKey] = useState(0)

  useEffect(() => {
    const controller = new AbortController()

    async function cargar() {
      try {
        const res = await fetch(url, { signal: controller.signal })
        if (!res.ok) throw new Error(`HTTP ${res.status}`)
        const json = (await res.json()) as T
        setState({ data: json, loading: false, error: null })
      } catch (err) {
        if (err instanceof DOMException && err.name === 'AbortError') return
        setState({
          data: null,
          loading: false,
          error: err instanceof Error ? err.message : 'Error desconocido',
        })
      }
    }

    cargar()
    return () => controller.abort()
  }, [url, reloadKey])

  const refetch = useCallback(() => {
    setState((s) => ({ ...s, loading: true, error: null }))
    setReloadKey((k) => k + 1)
  }, [])

  return { ...state, refetch }
}
```

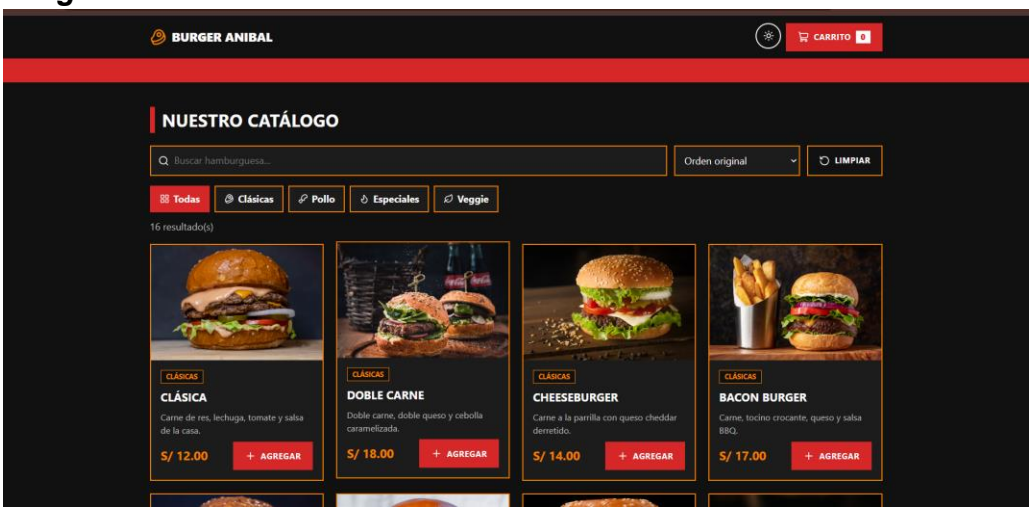
- Manejo de Ciclo de Vida y Limpieza:AbortController vincula la señal HTTP para cancelar peticiones pendientes cuando el componente se desmonta o cuando el efecto se vuelve a ejecutar.El uso de un array de dependencias vacío [ ] bajo StrictMode no garantiza una única ejecución en desarrollo; la función de limpieza previene conflictos y fugas de memoria.

- Estados Asociados:data: Almacena la información del catálogo obtenida de /burgers.json.loading: Booleano indicador de estado de carga (muestra mensaje "Cargando catálogo...").error: Captura incidencias de red, habilitando un mecanismo de reintento ("Reintentar").

Estado de carga de renderizado



Carga exitosa



PASO 3: ARQUITECTURA DE ESTADOS GLOBALES CON useContext (TEMA, ALERTAS Y CARRITO)

3.1. Objetivo de la Actividad

Implementar la Context API separando lógicas por dominios específicos para evitar el problema de *prop drilling* y estructurar proveedores (*Providers*) de manera limpia.

3.2. Análisis del Código y Conceptos Clave

- **Jerarquía de Proveedores en main.tsx:**

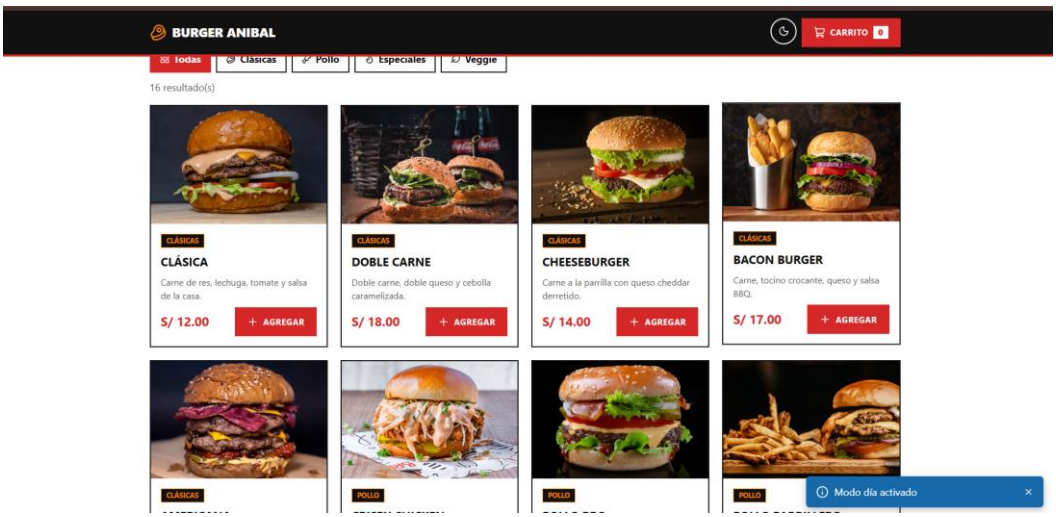
```tsx
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App'
import { ThemeProvider } from './context/ThemeContext'
import { ToastProvider } from './context/ToastContext'
import { CartProvider } from './context/CartContext'

createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <ThemeProvider>
      <ToastProvider>
        <CartProvider>
          <App />
        </CartProvider>
      </ToastProvider>
    </ThemeProvider>
  </StrictMode>,
)
```

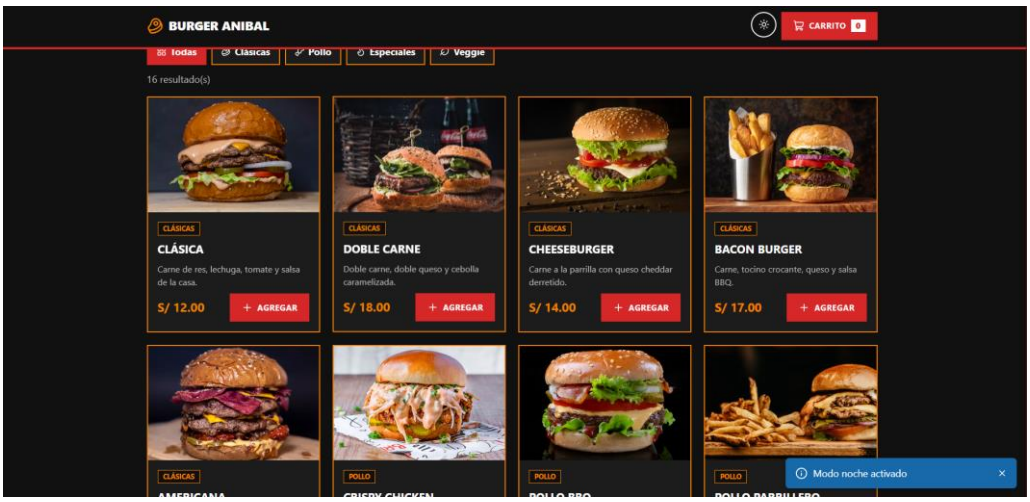
Dominios Implementados:

1. **Tema (ThemeContext.tsx):** Controla el modo día/noche, persiste la preferencia en localStorage y actualiza dinámicamente el atributo del DOM (document.documentElement.dataset.theme).
2. **Alertas (ToastContext.tsx):** Gestiona notificaciones flotantes globales mediante el método notify() consumido a través del hook useToast().
3. **Carrito (CartContext.tsx):** Administra los elementos seleccionados, conteos totales y operaciones de manipulación mediante useCart().

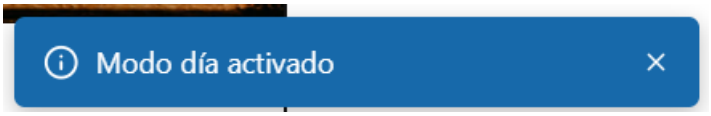
Thema de dia



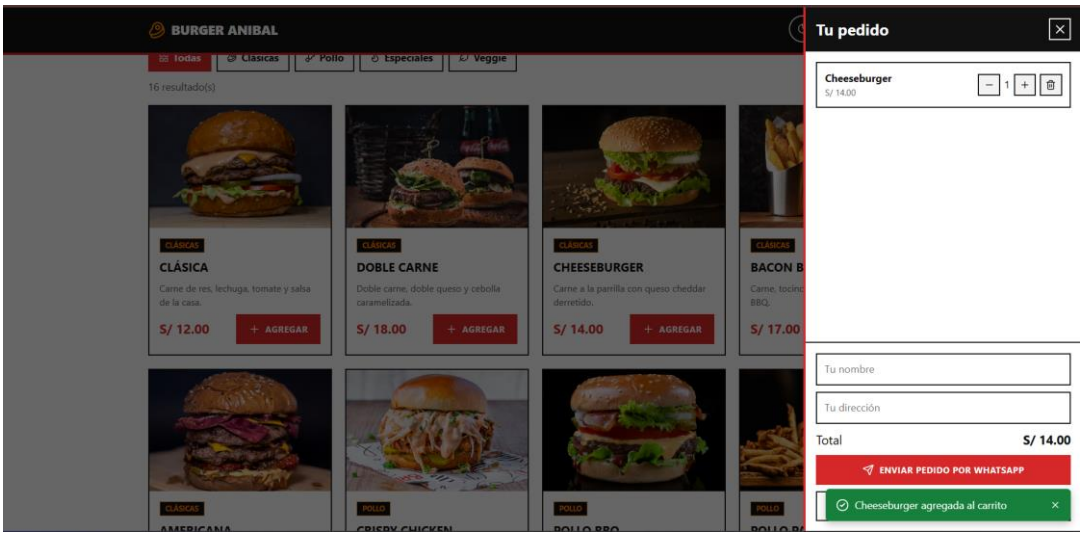
Thema de noche



Alerta



Carrito



PASO 4: OPTIMIZACIÓN DE RENDIMIENTO (useMemo, useCallback, useRef, memo)

4.1. Objetivo de la Actividad

Minimizar renderizados innecesarios aplicando técnicas de memorización de valores y funciones, así como referencias directas al DOM.

4.2. Análisis del Código y Conceptos Clave

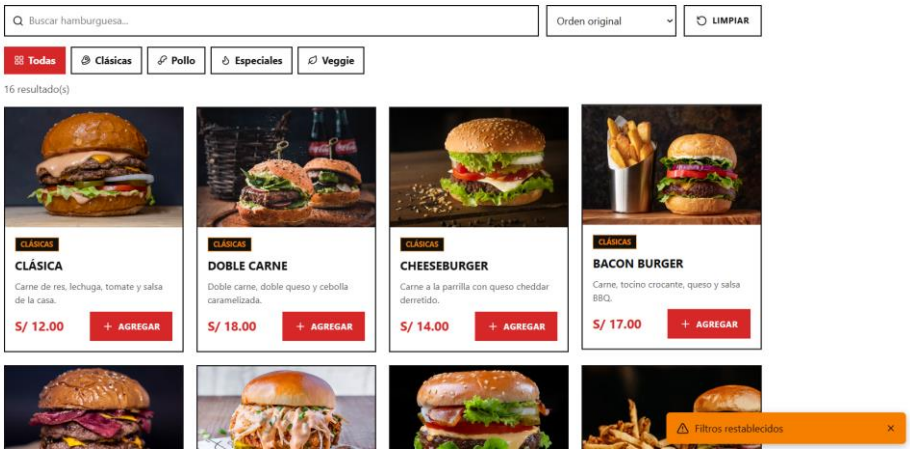
- **useMemo:** Memoriza colecciones filtradas y ordenadas en Catalog.tsx, recalculándose únicamente cuando varían burgers, query, cat o orden.

- **useCallback:** Estabiliza referencias de funciones clave (cambiarCategoria, limpiar, handleAdd, etc.) para prevenir re-renders innecesarios en componentes hijos optimizados.
- **useRef:** Vincula directamente una referencia al campo de búsqueda (inputRef) permitiendo enfocarlo mediante inputRef.current?.focus() de forma imperativa sin disparar renderizados redundantes.
- **memo:** Envuelve componentes como BurgerCard.tsx para evitar que se actualicen si sus *props* se mantienen intactas.

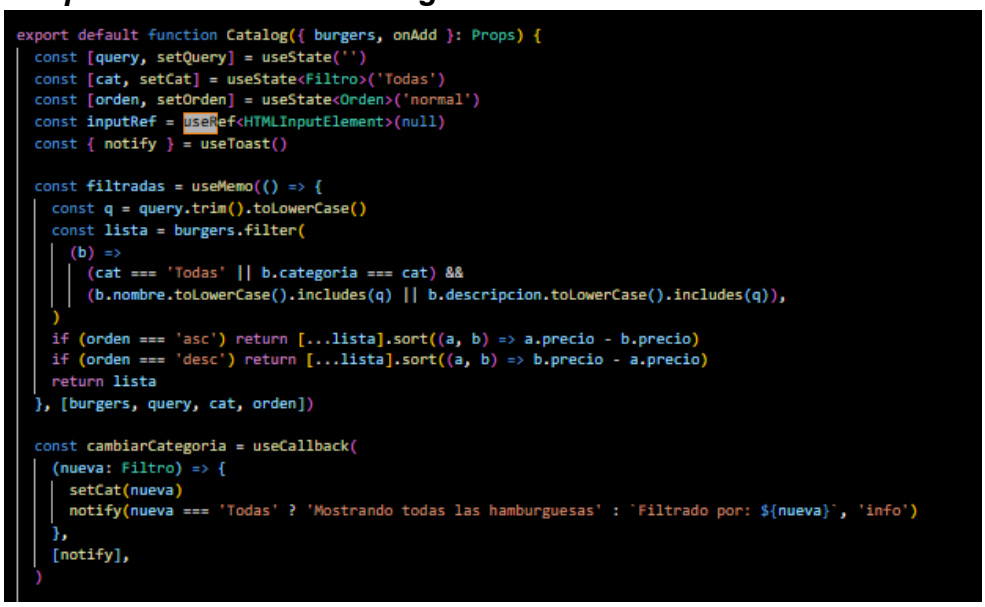
Busqueda



Limpiar



Scrip de mostrar las amburguesas UseRef



```tsx
export default function Catalog({ burgers, onAdd }: Props) {
  const [query, setQuery] = useState('')
  const [cat, setCat] = useState<Filtro>('Todas')
  const [orden, setOrden] = useState<Orden>('normal')
  const inputRef = useRef<HTMLInputElement>(null)
  const { notify } = useToast()

  const filtradas = useMemo(() => {
    const q = query.trim().toLowerCase()
    const lista = burgers.filter(
      (b) =>
        (cat === 'Todas' || b.categoria === cat) &&
        (b.nombre.toLowerCase().includes(q) || b.descripcion.toLowerCase().includes(q)),
    )
    if (orden === 'asc') return [...lista].sort((a, b) => a.precio - b.precio)
    if (orden === 'desc') return [...lista].sort((a, b) => b.precio - a.precio)
    return lista
  }, [burgers, query, cat, orden])

  const cambiarCategoria = useCallback(
    (nueva: Filtro) => {
      setCat(nueva)
      notify(nueva === 'Todas' ? 'Mostrando todas las hamburguesas' : `Filtrado por: ${nueva}`, 'info')
    },
    [notify],
  )
```

PASO 5: HOOKS PERSONALIZADOS Y GESTIÓN DE ESTADO COMPLEJO (useReducer)

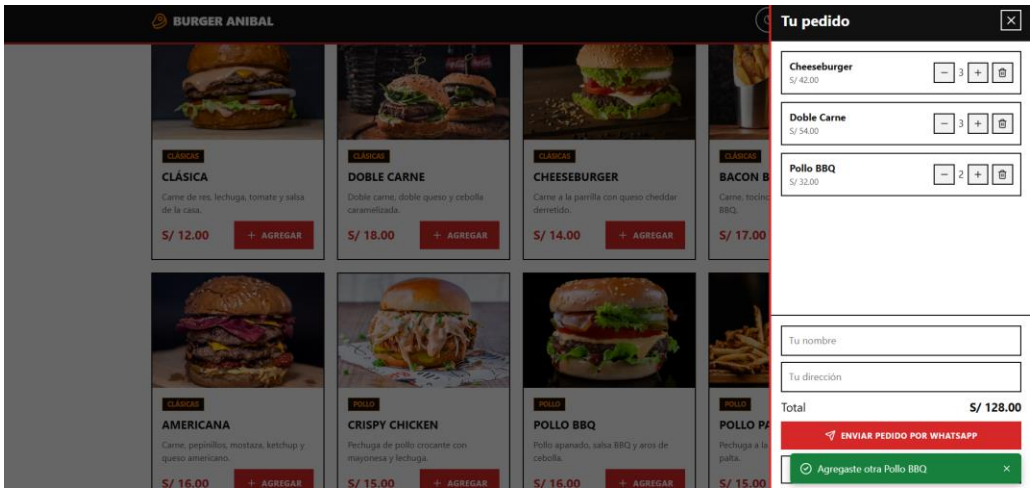
5.1. Objetivo de la Actividad

Encapsular lógica reutilizable en un Hook personalizado (useFetch) cumpliendo estrictamente con las reglas de los Hooks, y manejar la lógica de negocio compleja del carrito utilizando useReducer.

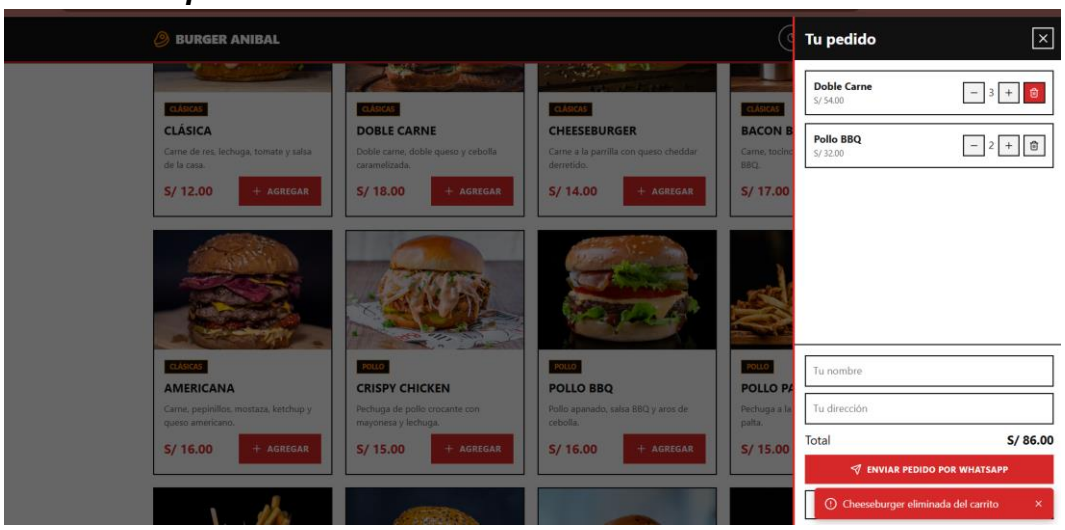
5.2. Análisis del Código y Conceptos Clave

- **Hook Personalizado (useFetch.ts):**
  - Extrae toda la lógica de obtención de datos, manejo de errores y cancelación.
  - Respeta las reglas fundamentales de los Hooks (llamado exclusivo en el nivel superior, sin condicionales ni bucles).
- **Gestión de Estado con useReducer (CartContext.tsx):**
  - Centraliza las operaciones del carrito (add, inc, dec, remove, clear) en una función reductora pura.
  - Garantiza la inmutabilidad del estado utilizando métodos funcionales (map, filter) y el operador de propagación (...).
- **Integración con Canales Externos:**
  - Generación automatizada de pedidos estructurados para su envío directo a través de WhatsApp con el detalle de productos y montos totales.

Productos al carrito



Eliminando productos del Carrito



Agregando o decremento o eliminado

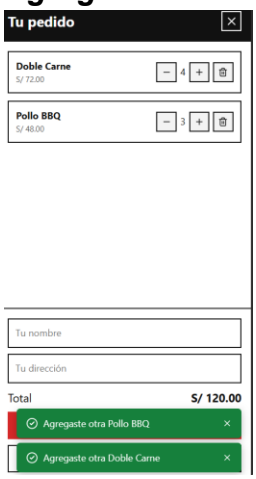
```
const CartContext = createContext<CartCtx | null>(null)

export function CartProvider({ children }: { children: ReactNode }) {
  const [items, dispatch] = useReducer(reducer, [])

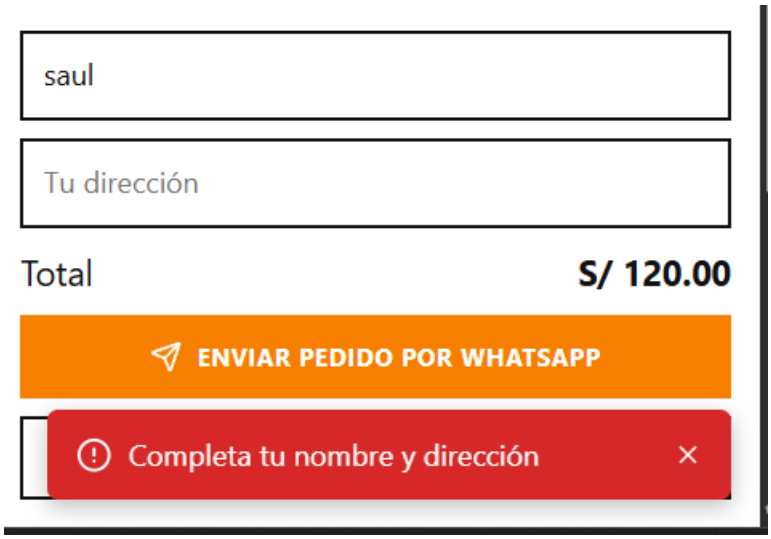
  const add = useCallback((burger: Burger) => dispatch({ type: 'add', burger }), [])
  const inc = useCallback((id: number) => dispatch({ type: 'inc', id }), [])
  const dec = useCallback((id: number) => dispatch({ type: 'dec', id }), [])
  const remove = useCallback((id: number) => dispatch({ type: 'remove', id }), [])
  const clear = useCallback(() => dispatch({ type: 'clear' }), [])

  const value = useMemo(
```

Agregando o disminuyendo



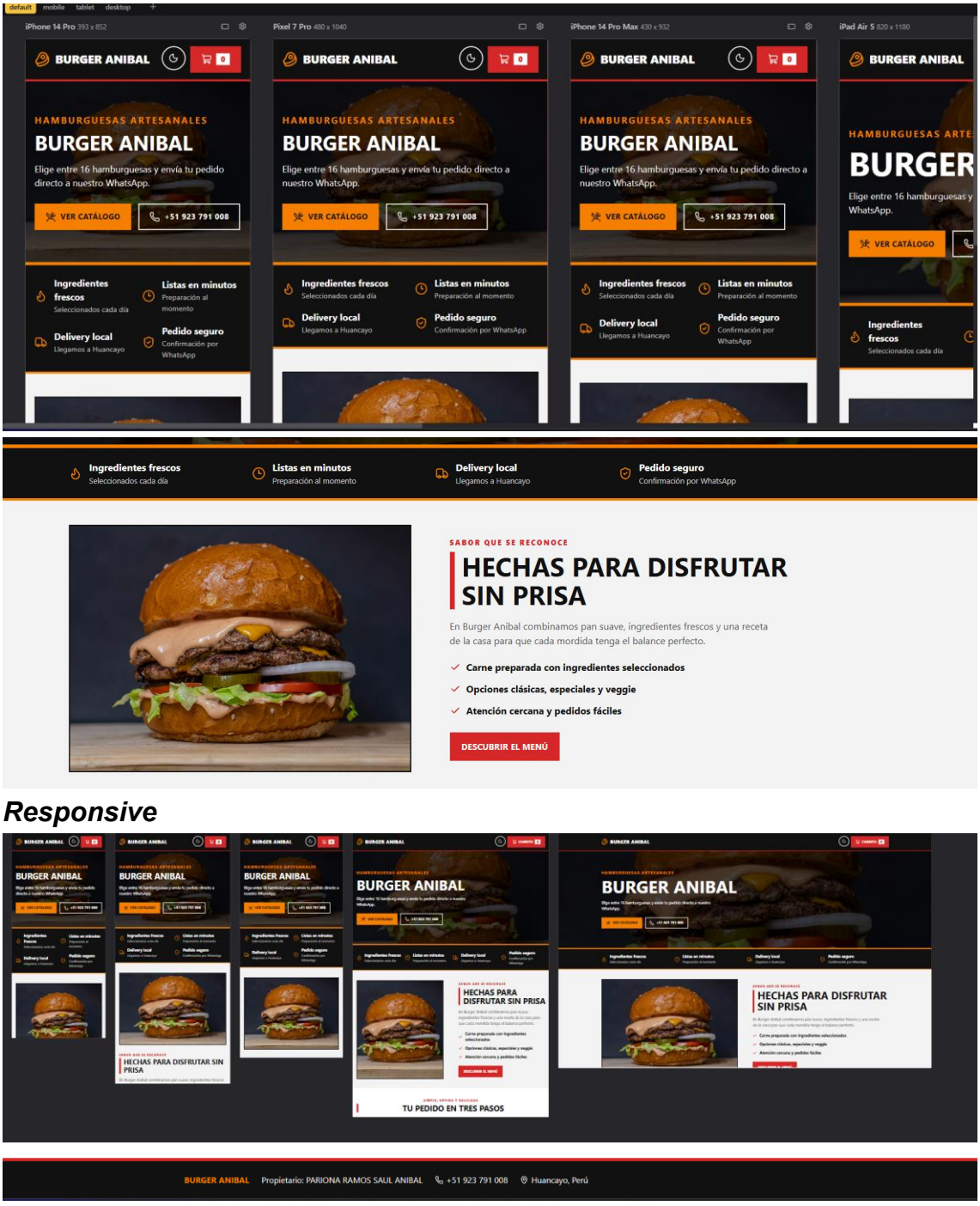
Formulario de validación



Pedido llegado a mi watsap



- Añada capturas de la **Página web funcional tipo SPA** (Single Page Application)implementada de manera local



Responsive

- Proporcione el enlace de Github de su proyecto

  https://github.com/saulPariona/Semana-7-AGuilar.git

Adicional: crear una video corto explicando los 5 pasos del ejercicio subirlo a Google drive y compartir con el correo miguelaguilaruncpfis@gmail.com para su revisión posterior.